UNIVERSITÀ
DEGLI STUDI DI TRIESTE

Data WareHouse Case Study
AA – 2024-25

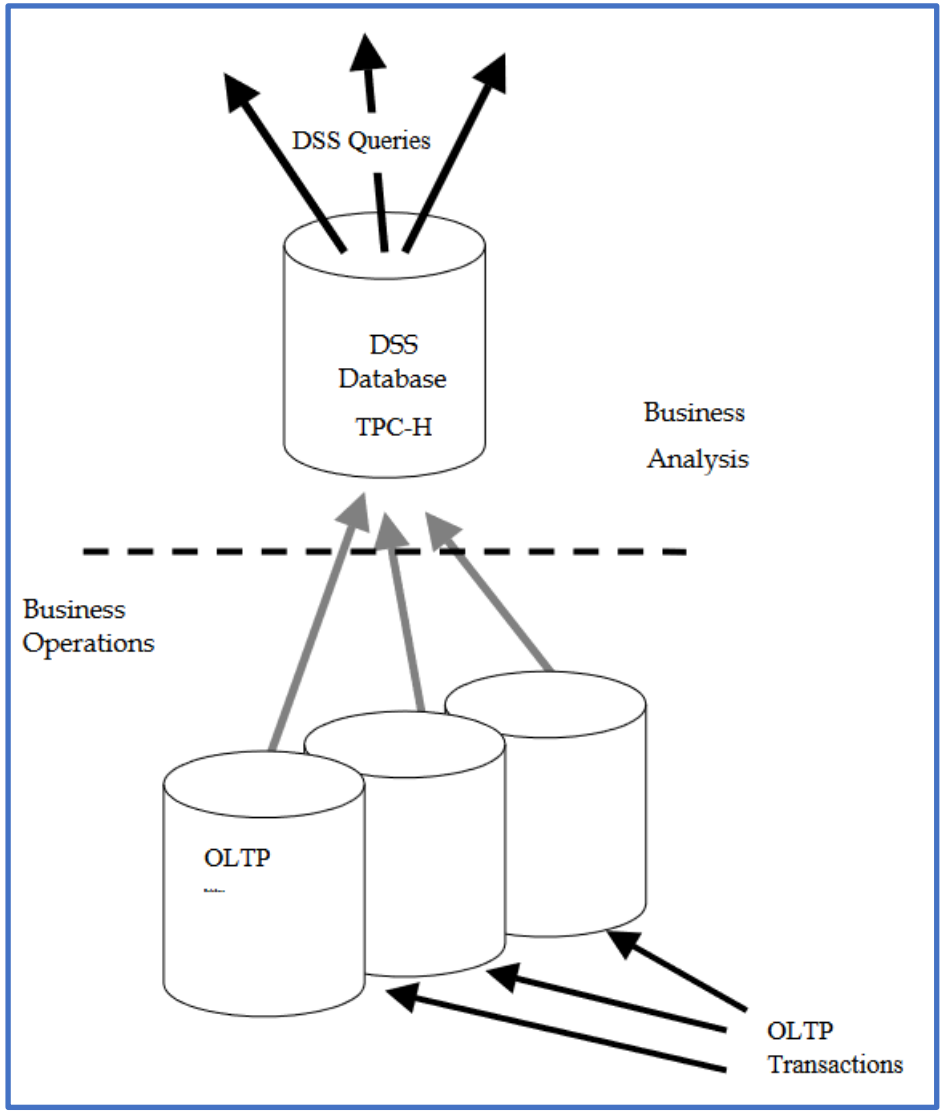
Prof. A. Peron

TPC-H

- A benchmark for decision support.
- www.tpc.org
- The TPC Benchmark™H (TPC-H) is a decision support benchmark. It consists of a suite of business oriented ad-hoc queries and concurrent data modifications.
- The queries and the data populating the database have been chosen to have broad industry-wide relevance while maintaining a sufficient degree of ease of implementation.
- This benchmark illustrates decision support systems that:
  - Examine large volumes of data;
  - Execute queries with a high degree of complexity;
  - Give answers to critical business questions.

TPC-H

- TPC Benchmark™ H is comprised of a set of business queries designed to exercise system functionalities in a manner representative of complex business analysis applications.
- These queries have been given a realistic context, portraying the activity of a wholesale supplier.
- TPC-H does not represent the activity of any particular business segment, but rather any industry which must manage sell, or distribute a product worldwide (e.g., car rental, food distribution, parts, suppliers, etc.).
- It includes:
  - a logical schema
  - a set of queries
  - a scalable set of data

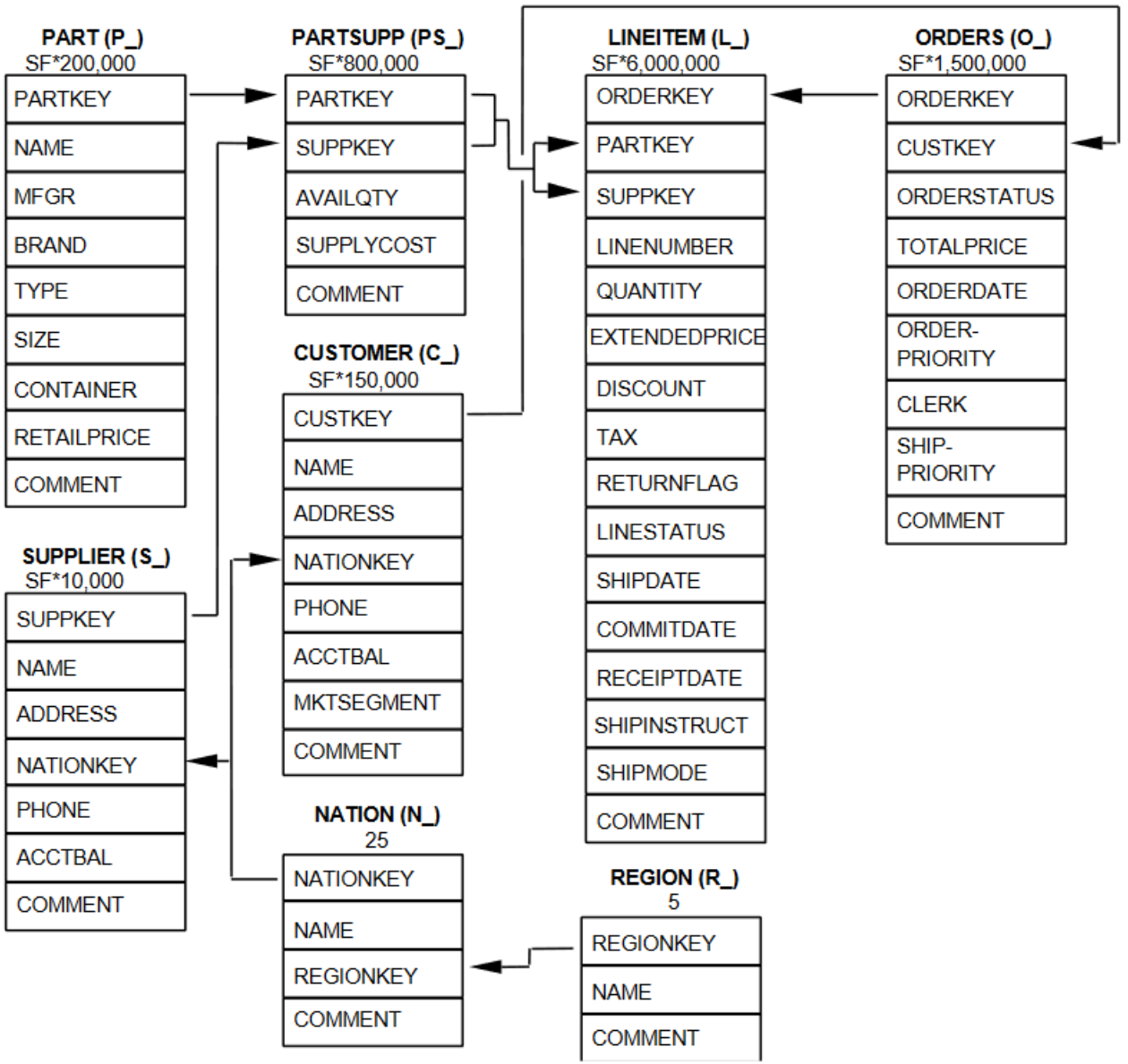


TPC-H - Schema

SF is Scale Factor * Number of rows

→

One-to-many association



TPC-H - Queries

- These selected queries provide answers to the following classes of business analysis:
  - Pricing and promotions;
  - Supply and demand management;
  - Profit and revenue management;
  - Customer satisfaction study;
  - Market share study;
  - Shipping management.
- The queries that have been selected exhibit the following characteristics:
  - They have a high degree of complexity;
  - They use a variety of access;
  - They are of an ad hoc nature;
  - They examine a large percentage of the available data;
  - They all differ from each other;
  - They contain query parameters that change across query executions.

TPC-H – Benchmark download

- **Metrics for a DB with scale factor 1**

Table Name	Cardinality (in rows)	Length (in bytes) of Typical[2] Row	Typical[2] Table Size (in MB)
SUPPLIER	10,000	159	2
PART	200,000	155	30
PARTSUPP	800,000	144	110
CUSTOMER	150,000	179	26
ORDERS	1,500,000	104	149
LINEITEM[3]	6,001,215	112	641
NATION[1]	25	128	< 1
REGION[1]	5	124	< 1
Total	8,661,245		956

[1] Fixed cardinality: does not scale with SF.

[2] Typical lengths and sizes given here are examples, not requirements, of what could result from an implementation (sizes do not include storage/access overheads).

[3] The cardinality of the LINEITEM table is not a strict multiple of SF since the number of lineitems in an order is chosen at random with an average of four (see Clause 4.2.5.2).

TPC-H – Benchmark download

- The benchmark can be downloaded at site www.tpc.org
- Complete documentation of the benchmark
- https://www.tpc.org/tpc_documents_current_versions/pdf/tpc-h_v3.0.1.pdf
- The documentation contains:
- A complete specification of the logical schema (relational schemata and data types).
- A set of predefined parametric queries.
- The package for creating the set of data can be obtained at site
- https://www.tpc.org/tpc_documents_current_versions/current_specifications5.asp
- The package allows to generate data parametric with respect to a Scale Factor

TPC-H – Benchmark download

- **Metrics for supported scale factor**

Table 4: LINEITEM Cardinality shows the cardinality of the LINEITEM table at all authorized scale factors.

Table 4: LINEITEM Cardinality

Scale Factor (SF)	Cardinality of LINEITEM Table
1	6001215
10	59986052
30	179998372
100	600037902
300	1799989091
1000	5999989709
3000	18000048306
10000	59999994267
30000	179999978268
100000	599999969200

TPC-H – Benchmark download

- **Download the dbgen package.**
- **Implement the DB in the DBMS PostgreSQL following the specification (structure and datatypes)**
- **Populate the DB using data generated by dbgen (csv files)**

A SF = 10 is suggested

Students attending the tutoring lessons has already collected all the data.

- Collect some table statistics
- Number of rows
- Table size
- Number of distinct values for each attribute
- MinValue and MaxValue for each meaningful attribute.

TPC-H queries

- The queries can be found in the document

https://www.tpc.org/tpc_documents_current_versions/pdf/tpc-h_v3.0.1.pdf

All the queries in the document are eligible:

Groups of one person: 4 queries

Groups of two people: 6 queries

Groups of three people: 9 queries.

Each group has to notify to adriano.peron@units.it the composition of the group and the subset of chosen queries.

Since all the queries should be chosen the set of proposed queries can be modified at notification time.

Cost model

Entities:

- $S = \{ Q_1, \dots, Q_k \}$ Set of queries
- $I = \{ I_1, \dots, I_n \}$ Set of indexes
- $M = \{ M_1, \dots, M_m \}$ Set of materialized views.
- $Size(.)$ Storage requirements for a structure
- $Time(.)$ Time for execution/creation/updating
- $N_{exec}(.)$ Number of executions in a period

Constraints:

For all i, $Time(Q_i) \leq TQ_i$ (response constraint)

$$\sum_{i=1}^{n} Size(I_i) + \sum_{i=1}^{m} Size(M_i) \leq MaxSize$$

(storage constraint)

Cost model

Cost:

$$\sum_{i=1}^{n} Time(I_i) + \sum_{i=1}^{m} Time(M_i) + \sum_{i=1}^{k} N_{exec}(Q_i) \times Time(Q_i)$$

1) $\sum_{i=1}^{n} Time(I_i)$ **(time for build/update indexes)**
2) $\sum_{i=1}^{m} Time(M_i)$ **(time for build/refresh materialized views)**
3) $\sum_{i=1}^{k} N_{exec}(Q_i) \times Time(Q_i)$ **(time for executing queries)**

Query optimization

The optimization should follow the following steps:

1) **Compute size and time for executing the queries without additional structure support.**
2) **Introduce a suitable set of indexes recording space and time cost for building indexes. Check that the indexes are actually used using the explain-analyze option.**
3) **Try to force different forms of join implementations acting on the environment variables.**
4) **Discharge indexes and introduce materialized views. Record time for the query execution with materialized views.**
5) **Consider a mixed approach which combines indexes which are been proved to be useful and materialized views (indexes can be added also to materialized views if it is the case).**
6) **For each step present the cost w.r. to the adopted cost model.**

Query optimization

Optional steps:

1) **Optionally consider the rewriting of the query (rewriting must preserve the result of the query)**
2) **Optionally consider fragmentation**

Optimization constraints

Remarks:

1) **The optimization activity is for the whole set of chosen queries and not for a single query.**
2) **One should not just materialize a query in the set.**
3) **One should not consider materializations which uses concrete instantiations of the parameters.**
4) **A materialization should support at least two queries**
5) **Use only indexes which are useful.**
6) **The total size of chosen indexes and materialized views should be comparable with the size of the original database.**